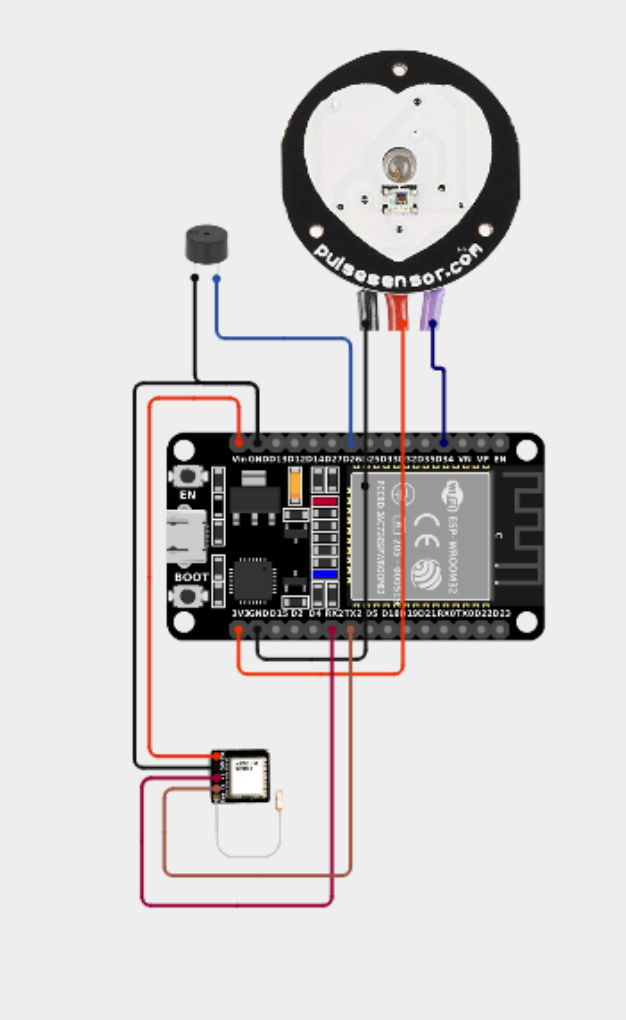
Heart Rate Monitor System with email and GPS

Circuit



Components used

1. Esp32
2. Questar GPS module
3. 5V buzzer
4. Heart rate sensor

Code

```
#include <Arduino.h>

#include <Wire.h>
```

```
#include <WiFi.h>
#include <WebServer.h>
#include <ESP_Mail_Client.h>
#include <TinyGPSPlus.h>

// ========================
// PIN DEFINITIONS (ESP32)
// ========================
#define HEART_RATE_PIN   34   // Analog input (use 34/35/36/39 — input-only ADC pins)
#define BUZZER_PIN       26   // Buzzer GPIO
#define GPS_RX_PIN       16   // GPS TX → ESP32 RX2
#define GPS_TX_PIN       17   // GPS RX → ESP32 TX2

// ========================
// WIFI & EMAIL SETTINGS
// ========================
#define WIFI_SSID       "Projects"
#define WIFI_PASSWORD   "12345678@"
#define SMTP_HOST       "smtp.gmail.com"
#define SMTP_PORT       587
#define AUTHOR_EMAIL    "29iamthenextironman@gmail.com"
#define AUTHOR_PASSWORD "tkfotjremhlymxzr"

// ========================
// CONFIGURABLE SETTINGS
// ========================
String recipientEmail = "sujith20040529@gmail.com";
int    hrThreshold    = 55;
```

```
float  bpmMultiplier  = 2.0;

// ========================
// HEART RATE (ANALOG)
// ========================
#define SAMPLE_INTERVAL_MS  2      // Read every 2 ms → 500 Hz
#define PEAK_THRESHOLD     2200    // Raw ADC value above which a beat is detected (tune this!)
#define MIN_BEAT_INTERVAL   300    // Minimum ms between beats (= max 200 BPM)
#define SIGNAL_HIGH_THRESH  2000   // ADC value above which finger is considered detected

const byte RATE_SIZE = 4;
byte  rates[RATE_SIZE];
byte  rateSpot     = 0;
long  lastBeat     = 0;
float beatsPerMinute = 0;
int   beatAvg      = 0;
bool  fingerDetected = false;
bool  lastAbovePeak  = false;

unsigned long lastSampleTime = 0;

// ========================
// GPS
// ========================
TinyGPSPlus gps;
HardwareSerial gpsSerial(2);   // UART2
double latitude    = 0.0;
double longitude   = 0.0;
bool   gpsValid    = false;
```

```
unsigned long lastGPSUpdate = 0;
const unsigned long GPS_TIMEOUT = 5000;

// ========================
// WEB SERVER
// ========================
WebServer server(80);

// ========================
// EMAIL
// ========================
SMTPSession smtp;
bool emailSent = false;
unsigned long lastEmailTime = 0;
const unsigned long EMAIL_COOLDOWN = 5000;

// ========================
// FORWARD DECLARATIONS
// ========================
void beepBuzzer(int duration);
void alertBuzzer();
void updateGPS();
void sendEmail(float heartRate);
void smtpCallback(SMTP_Status status);
void handleRoot();
void handleData();
void handleSetThreshold();
void handleSetMultiplier();
```

```
void handleTestBuzzer();

void handleSetEmail();

void handleSendTestEmail();


// ========================

// WEB PAGE

// ========================

const char MAIN_page[] = R"=====(

<!DOCTYPE html>

<html lang="en">

<head>

<meta charset="UTF-8">

<meta name="viewport" content="width=device-width, initial-scale=1.0">

<title>❤️ Heart Rate Monitor</title>

<style>

 body{font-family:Arial,sans-serif;background:#1a1a2e;color:#eee;margin:0;padding:20px;}

 .card{background:#16213e;border-radius:15px;padding:20px;margin:15px auto;max-width:500px;box-shadow:0 4px 15px rgba(0,0,0,0.3);}

 .bpm-display{font-size:72px;font-weight:bold;color:#e94560;text-align:center;}

 .label{font-size:14px;color:#aaa;text-align:center;margin-bottom:5px;}

 .status{font-size:18px;text-align:center;color:#4ecca3;}

 input[type=number],input[type=email]{width:100%;padding:10px;border-radius:8px;border:1px solid #333;background:#0f3460;color:#eee;font-size:16px;box-sizing:border-box;margin:8px 0;}

 button{width:100%;padding:12px;border-radius:8px;border:none;background:#e94560;color:#fff;font-size:16px;cursor:pointer;margin-top:8px;}

 button:hover{background:#c73652;}

 .info-grid{display:grid;grid-template-columns:1fr 1fr;gap:10px;margin-top:10px;}

 .info-item{background:#0f3460;border-radius:10px;padding:12px;text-align:center;}

 .info-val{font-size:22px;font-weight:bold;color:#4ecca3;}
```

```
 .info-label{font-size:11px;color:#aaa;margin-top:4px;}
 h2{text-align:center;color:#e94560;margin-top:0;}
 a{color:#4ecca3;}
</style>
</head>
<body>
<div class="card">
 <h2>❤️ Heart Rate Monitor</h2>
 <div class="label">Average BPM</div>
 <div class="bpm-display" id="bpm">--</div>
 <div class="status" id="status">Waiting...</div>
</div>

<div class="card">
 <h2>⚙️ Alert Settings</h2>
 <label>Threshold (BPM)</label>
 <input type="number" id="thresholdInput" min="20" max="200" value="55">
 <button onclick="setThreshold()">Update Threshold</button>

 <label style="margin-top:15px;display:block;">BPM Multiplier</label>
 <input type="number" id="multiplierInput" min="0.1" max="10" step="0.1" value="2.0">
 <button onclick="setMultiplier()">Update Multiplier</button>

 <button onclick="testBuzzer()" style="background:#0f3460;margin-top:12px;">🔔 Test Buzzer</button>
</div>

<div class="card">
 <h2>📧 Email Settings</h2>
 <label>Recipient Email</label>
```

```
 <input type="email" id="emailInput" placeholder="Enter email...">
 <div class="label" id="currentEmail">Current: Loading...</div>
 <button onclick="setEmail()">Update Email</button>
 <button onclick="sendTestEmail()" style="background:#0f3460;margin-top:8px;">📨 Send Test Email</button>
</div>

<div class="card">
 <h2>📊 Live Stats</h2>
 <div class="info-grid">
  <div class="info-item"><div class="info-val" id="instantBpm">--</div><div class="info-label">Instant BPM</div></div>
  <div class="info-item"><div class="info-val" id="threshVal">--</div><div class="info-label">Threshold (BPM)</div></div>
  <div class="info-item"><div class="info-val" id="gpsStatus">--</div><div class="info-label">GPS Status</div></div>
  <div class="info-item"><div class="info-val" id="emailStatus">--</div><div class="info-label">Email Alert</div></div>
 </div>
 <div id="mapLink" style="text-align:center;margin-top:10px;"></div>
</div>

<script>
function fetchData(){
 fetch('/data').then(r=>r.json()).then(d=>{
  document.getElementById('bpm').textContent = d.finger ? d.avgBpm.toFixed(0) : '--';
  document.getElementById('status').textContent = d.finger ? (d.avgBpm > d.threshold ? '🚨 HIGH BPM ALERT!' : '✅ Normal') : '👆 Place finger on sensor';
  document.getElementById('instantBpm').textContent = d.finger ? d.bpm.toFixed(0) : '--';
  document.getElementById('threshVal').textContent = d.threshold;
  document.getElementById('gpsStatus').textContent = d.gpsValid ? '✅ Fixed' : '⚠️ No Fix';
```

```
   document.getElementById('emailStatus').textContent = d.emailSent ? '✅ Sent' : '⏳ Waiting';
   document.getElementById('currentEmail').textContent = 'Current: ' + d.recipientEmail;
   document.getElementById('thresholdInput').value = d.threshold;
   document.getElementById('multiplierInput').value = d.multiplier;
   if(d.gpsValid){
    document.getElementById('mapLink').innerHTML = '<a href="https://www.google.com/maps?q='+d.lat+','+d.lng+'" target="_blank">📍 View Location on Map</a>';
   } else {
    document.getElementById('mapLink').innerHTML = '';
   }
  });
}
function setThreshold(){
  let v = document.getElementById('thresholdInput').value;
  fetch('/setThreshold?value='+v).then(r=>r.text()).then(t=>alert(t==='OK'?'Threshold updated!':t));
}
function setMultiplier(){
  let v = document.getElementById('multiplierInput').value;
  fetch('/setMultiplier?value='+v).then(r=>r.text()).then(t=>alert(t==='OK'?'Multiplier updated!':t));
}
function testBuzzer(){
  fetch('/testBuzzer');
}
function setEmail(){
  let v = document.getElementById('emailInput').value;
  fetch('/setEmail?value='+encodeURIComponent(v)).then(r=>r.text()).then(t=>alert(t==='OK'?'Email updated!':t));
}
```

```
function sendTestEmail(){
 alert('Sending test email...');
 fetch('/sendTestEmail').then(r=>r.text()).then(t=>alert(t==='OK'?'Test email sent!':t));
}
setInterval(fetchData, 1000);
fetchData();
</script>
</body>
</html>
)=====";

// ========================
// SETUP
// ========================
void setup() {
 Serial.begin(115200);
 Serial.println("Initializing ESP32 Heart Rate Monitor...");

 // Buzzer
 pinMode(BUZZER_PIN, OUTPUT);
 digitalWrite(BUZZER_PIN, LOW);
 beepBuzzer(100);

 // GPS on UART2
 gpsSerial.begin(9600, SERIAL_8N1, GPS_RX_PIN, GPS_TX_PIN);
 Serial.println("GPS UART2 initialized");

 // WiFi
 WiFi.begin(WIFI_SSID, WIFI_PASSWORD);
```

```
  Serial.print("Connecting to WiFi");
  while (WiFi.status() != WL_CONNECTED) {
    delay(500);
    Serial.print(".");
  }
  Serial.println("\nWiFi connected! IP: " + WiFi.localIP().toString());

  // Web server routes
  server.on("/",            handleRoot);
  server.on("/data",        handleData);
  server.on("/setThreshold",  handleSetThreshold);
  server.on("/setMultiplier", handleSetMultiplier);
  server.on("/testBuzzer",    handleTestBuzzer);
  server.on("/setEmail",      handleSetEmail);
  server.on("/sendTestEmail", handleSendTestEmail);
  server.begin();
  Serial.println("Web server started!");

  // ADC setup
  analogReadResolution(12);      // 12-bit: 0–4095
  analogSetAttenuation(ADC_11db); // Full 0–3.3V range

  // Ready beeps
  beepBuzzer(100); delay(100); beepBuzzer(100);
  Serial.println("Place finger on analog pulse sensor.");
  Serial.printf("Alert threshold: %d BPM\n", hrThreshold);
}

// =======================
```

```
// LOOP
// =======================
void loop() {
 server.handleClient();
 updateGPS();

 unsigned long now = millis();
 if (now - lastSampleTime >= SAMPLE_INTERVAL_MS) {
  lastSampleTime = now;

  int rawValue = analogRead(HEART_RATE_PIN);

  // Finger detection
  fingerDetected = (rawValue > SIGNAL_HIGH_THRESH);

  // Peak detection (rising edge)
  bool abovePeak = (rawValue > PEAK_THRESHOLD);
  if (abovePeak && !lastAbovePeak) {
   long delta = now - lastBeat;
   if (delta > MIN_BEAT_INTERVAL) {
    lastBeat = now;
    beatsPerMinute = 60000.0 / delta;

    if (beatsPerMinute > 20 && beatsPerMinute < 255) {
     rates[rateSpot++] = (byte)beatsPerMinute;
     rateSpot %= RATE_SIZE;
     beatAvg = 0;
     for (byte x = 0; x < RATE_SIZE; x++) beatAvg += rates[x];
     beatAvg /= RATE_SIZE;
```

```
    }
   }
  }
  lastAbovePeak = abovePeak;

  // Serial debug every 500ms
  static unsigned long lastPrint = 0;
  if (now - lastPrint >= 500) {
   lastPrint = now;
   Serial.printf("ADC=%d | BPM=%.1f | AvgBPM=%.1f | Finger=%s",
    rawValue, beatsPerMinute, (float)(beatAvg * bpmMultiplier),
    fingerDetected ? "YES" : "NO");
   if (gpsValid) Serial.printf(" | GPS: %.6f,%.6f", latitude, longitude);
   else       Serial.print(" | GPS: No fix");
   Serial.println();
  }
}

// Alert logic
if (!fingerDetected) {
  emailSent = false;
} else {
  float displayBpm = beatAvg * bpmMultiplier;
  if (displayBpm > hrThreshold) {
   if (!emailSent) {
    unsigned long currentTime = millis();
    if (currentTime - lastEmailTime > EMAIL_COOLDOWN) {
     Serial.println("\n🚨 HIGH BPM — Sending alert email!");
```

```
        alertBuzzer();
        sendEmail(displayBpm);
        emailSent = true;
        lastEmailTime = currentTime;
       }
     }
    }
    if (beatAvg > 0 && (beatAvg * bpmMultiplier) < (hrThreshold - 5)) {
     emailSent = false;
    }
  }
}

// ========================
// BUZZER
// ========================
void beepBuzzer(int duration) {
  digitalWrite(BUZZER_PIN, HIGH);
  delay(duration);
  digitalWrite(BUZZER_PIN, LOW);
}

void alertBuzzer() {
  for (int i = 0; i < 3; i++) {
    digitalWrite(BUZZER_PIN, HIGH); delay(300);
    digitalWrite(BUZZER_PIN, LOW);  delay(150);
  }
}
```

```
// ========================
// GPS
// ========================
void updateGPS() {
  while (gpsSerial.available() > 0) {
    char c = gpsSerial.read();
    if (gps.encode(c)) {
      if (gps.location.isValid()) {
        latitude  = gps.location.lat();
        longitude = gps.location.lng();
        gpsValid  = true;
        lastGPSUpdate = millis();
      }
    }
  }
  if (gpsValid && (millis() - lastGPSUpdate > GPS_TIMEOUT)) {
    gpsValid = false;
  }
}

// ========================
// WEB SERVER HANDLERS
// ========================
void handleRoot() {
  server.send(200, "text/html", MAIN_page);
}
```

```
void handleData() {
  String json = "{";
  json += "\"bpm\":"         + String(beatsPerMinute)         + ",";
  json += "\"avgBpm\":"      + String(beatAvg * bpmMultiplier)  + ",";
  json += "\"threshold\":"   + String(hrThreshold)            + ",";
  json += "\"multiplier\":"  + String(bpmMultiplier, 1)       + ",";
  json += "\"finger\":"      + String(fingerDetected ? "true" : "false") + ",";
  json += "\"gpsValid\":"    + String(gpsValid     ? "true" : "false") + ",";
  json += "\"lat\":"         + String(latitude,  6)           + ",";
  json += "\"lng\":"         + String(longitude, 6)           + ",";
  json += "\"emailSent\":"   + String(emailSent  ? "true" : "false") + ",";
  json += "\"recipientEmail\":\"" + recipientEmail + "\"";
  json += "}";
  server.send(200, "application/json", json);
}

void handleSetThreshold() {
  if (server.hasArg("value")) {
    int v = server.arg("value").toInt();
    if (v >= 20 && v <= 200) {
      hrThreshold = v;
      emailSent   = false;
      Serial.printf("Threshold → %d BPM\n", hrThreshold);
      beepBuzzer(50);
      server.send(200, "text/plain", "OK");
    } else {
      server.send(400, "text/plain", "Invalid threshold (20-200)");
```

```
    }
  } else {
    server.send(400, "text/plain", "Missing value");
  }
}

void handleSetMultiplier() {
  if (server.hasArg("value")) {
    float v = server.arg("value").toFloat();
    if (v >= 0.1 && v <= 10.0) {
      bpmMultiplier = v;
      emailSent     = false;
      Serial.printf("Multiplier → %.1f×\n", bpmMultiplier);
      beepBuzzer(50);
      server.send(200, "text/plain", "OK");
    } else {
      server.send(400, "text/plain", "Invalid multiplier (0.1-10.0)");
    }
  } else {
    server.send(400, "text/plain", "Missing value");
  }
}

void handleTestBuzzer() {
  Serial.println("Buzzer test from web!");
  beepBuzzer(200); delay(100); beepBuzzer(200);
  server.send(200, "text/plain", "OK");
}
```

```
void handleSetEmail() {
  if (server.hasArg("value")) {
    String e = server.arg("value");
    if (e.indexOf('@') > 0 && e.indexOf('.') > 0) {
      recipientEmail = e;
      Serial.printf("Recipient email → %s\n", recipientEmail.c_str());
      beepBuzzer(50);
      server.send(200, "text/plain", "OK");
    } else {
      server.send(400, "text/plain", "Invalid email format");
    }
  } else {
    server.send(400, "text/plain", "Missing value");
  }
}

// ========================
// SHARED EMAIL BUILDER
// ========================
static void buildAndSend(bool isTest, float heartRate) {
  if (WiFi.status() != WL_CONNECTED) {
    WiFi.begin(WIFI_SSID, WIFI_PASSWORD);
    int attempts = 0;
    while (WiFi.status() != WL_CONNECTED && attempts < 20) {
      delay(500); Serial.print("."); attempts++;
    }
    if (WiFi.status() != WL_CONNECTED) {
```

```
    Serial.println("WiFi reconnect failed!"); return;
  }
}

smtp.debug(1);
smtp.callback(smtpCallback);

Session_Config config;
config.server.host_name  = SMTP_HOST;
config.server.port       = SMTP_PORT;
config.login.email       = AUTHOR_EMAIL;
config.login.password    = AUTHOR_PASSWORD;
config.login.user_domain = "";
config.time.ntp_server   = F("pool.ntp.org,time.nist.gov");
config.time.gmt_offset   = 5 * 3600 + 30 * 60;
config.time.day_light_offset = 0;

SMTP_Message message;
message.sender.name  = F("Heart Rate Monitor");
message.sender.email = AUTHOR_EMAIL;
message.addRecipient(F("User"), recipientEmail.c_str());

if (isTest) {
  message.subject = F("✅ Test Email - Heart Rate Monitor");
} else {
  message.subject = F("⚠️ High Heart Rate Alert!");
  message.priority = esp_mail_smtp_priority::esp_mail_smtp_priority_high;
}
```

```
String locationInfo = gpsValid
  ? "Lat: " + String(latitude,6) + "  Lng: " + String(longitude,6)
  : "GPS signal not available";
String locationLink = gpsValid
  ? "https://www.google.com/maps?q=" + String(latitude,6) + "," + String(longitude,6)
  : "";

// Plain text
String textMsg = isTest
  ? "Test email from Heart Rate Monitor.\n\n"
  : "High heart rate detected!\n\n";
textMsg += "Heart Rate : " + String(heartRate, 1) + " BPM\n";
textMsg += "Threshold  : " + String(hrThreshold) + " BPM\n";
textMsg += "Location   : " + locationInfo + "\n";
if (gpsValid) textMsg += "Maps       : " + locationLink + "\n";
textMsg += isTest
  ? "\nAlert system is working correctly!"
  : "\nPlease check on the patient immediately.";
message.text.content = textMsg.c_str();

// HTML
String accent = isTest ? "#4ecca3" : "#e94560";
String title  = isTest ? "✅ Test Email" : "⚠️ Heart Rate Alert";
String htmlMsg = "<div style='font-family:Arial,sans-serif;background:#1a1a2e;color:#eee;padding:20px;'>";
htmlMsg += "<div style='max-width:500px;margin:auto;background:#16213e;border-radius:15px;padding:20px;'>";
htmlMsg += "<h2 style='color:" + accent + ";text-align:center;'>" + title + "</h2>";
```

```
 if (!isTest) htmlMsg += "<p style='text-align:center;color:#e94560;font-weight:bold;'>High Heart Rate Detected!</p>";

 htmlMsg += "<div style='background:#0f3460;border-radius:10px;padding:15px;text-align:center;margin:15px 0;'>";

 htmlMsg += "<div style='font-size:48px;font-weight:bold;color:" + accent + ";'>" + String(heartRate,1) + " BPM</div>";

 htmlMsg += "<div style='color:#aaa;'>Threshold: " + String(hrThreshold) + " BPM</div></div>";

 htmlMsg += "<div style='background:#0f3460;border-radius:10px;padding:15px;margin:15px 0;'>";

 htmlMsg += "<h3 style='color:#4ecca3;margin:0 0 10px;'>📍 Location</h3>";

 if (gpsValid) {

  htmlMsg += "<p>Latitude: " + String(latitude,6) + "</p>";

  htmlMsg += "<p>Longitude: " + String(longitude,6) + "</p>";

  htmlMsg += "<a href='" + locationLink + "' style='color:#4ecca3;'>📍 View on Google Maps</a>";

 } else {

  htmlMsg += "<p style='color:#aaa;'>⚠️ GPS Not Fixed</p>";

 }

 htmlMsg += "</div>";

 if (!isTest) htmlMsg += "<p style='color:#e94560;font-weight:bold;text-align:center;'>⚡ Please check on the patient immediately!</p>";

 else       htmlMsg += "<p style='color:#4ecca3;text-align:center;'>✅ Alert system is working correctly!</p>";

 htmlMsg += "</div></div>";


 message.html.content         = htmlMsg.c_str();

 message.html.charSet         = "us-ascii";

 message.html.transfer_encoding = Content_Transfer_Encoding::enc_7bit;


 if (!smtp.connect(&config)) {

  config.server.port = 465;

  if (!smtp.connect(&config)) {

   Serial.println("SMTP connection failed on both ports!"); return;
```

```
  }
 }

 if (!MailClient.sendMail(&smtp, &message))
  Serial.println("❌ Email failed: " + smtp.errorReason());
 else {
  Serial.println("✅ Email sent!");
  alertBuzzer();
 }
 smtp.closeSession();
}

void handleSendTestEmail() {
 Serial.println("Test email from web!");
 buildAndSend(true, beatAvg * bpmMultiplier);
 server.send(200, "text/plain", "OK");
}

void sendEmail(float heartRate) {
 Serial.println("Sending alert email...");
 buildAndSend(false, heartRate);
}

// ========================
// SMTP CALLBACK
// ========================
void smtpCallback(SMTP_Status status) {
 Serial.println(status.info());
 if (status.success()) {
```

```
    ESP_MAIL_PRINTF("Sent: %d | Failed: %d\n",
      status.completedCount(), status.failedCount());
  }
}
```